

Deliverable report

"Vittorio Polci": Mat: 257817, vittorio.polci@unitn.it, GitRepo: <https://github.com/vittorio01/GPU-Computing-2025-257817>

Abstract—This deliverable discusses about the implementation of different parallel solutions of the SpVM algorithm between CSR matrices and a general vector, with a final benchmark that compares all solution together.

Index Terms—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

I. INTRODUCTION

The algorithm of sparse matrix-dense vector multiplication (SpMV) is an optimized version of the traditional matrix-vector multiplication designed to handle cases where the matrix is organized in a low density of non-zero entries. This technique is particularly useful in scenarios that involves very large sparse matrices because it allows to reduce the used space for storing that large structures and, at the same time, optimizes memory access patterns during computational steps.

Because the SpMV multiplication should handle matrices with millions or billions of not null elements, to obtain good results is necessary to use parallelization techniques on GPUs rather than sequential solution for CPUs.

II. PROBLEM STATEMENT

In the SpVM multiplication we consider a matrix coded in a sparse format (in this case the CSR format) and an input vector that has the same size of its columns number.

Each elements of the resulting vector is computed by performing the dot products of each not null elements of the matrix with the input vector and accumulating them in the corresponding row position. At this point the main objectives are:

- 1) Create a first sequential algorithm that performs the same operation on a sparse CSR matrix.
- 2) Parallelize the first CPU implementation with CUDA and compare them in terms of performances and resurces utilization.

A. Storage Format

The CSR for sparse matrices is a storage format that involves three different arrays to represent the information:

- The data array, that contains all not null terms of the original matrix.
- The column array, where for each element in the data vector specifies its column coordinate.
- The row array, which contains the start and the end pointers for each row to the data and column arrays. these two pair of pointers indicates which not null terms in the data array are contained in that specific row.

The size of the data and column arrays depends on the number of not null elements and the size of the row array depends on the number of the rows in the original matrix.

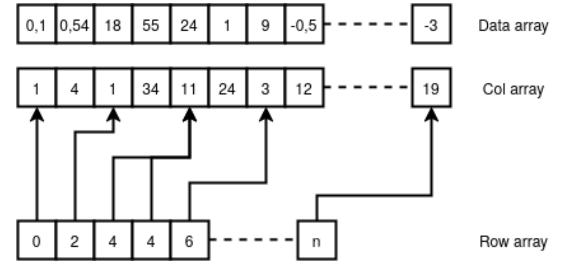


Fig. 1. Data organization in CSR structure

B. Parallelization

To improve the overall performance, the parallel implementations of the first algorithm are designed to be executed on Nvidia GPUs and programmed in CUDA. The CUDA language model permits to run a kernel on different concurrent threads, which are scheduled and executed by computational elements called CUDA cores.

To improve the performance and the organization of the parallel algorithms, CUDA architectures provides:

- *Warp organization*, where groups of 32 threads can synchronize and share data using predefined function for reductions.
- *Block organization*, where more warps can synchronize themselves using atomic operations, barriers and shared memory.
- *Shared memory*, which is divided in more blocks and used to improve data reuse, optimize the global memory transfers and other purposes.
- *Read only cache*, which is a L1 read only cache that can be assigned by the programmer for improving data transfer for casual and repetitive accesses.

III. STATE OF THE ART

There already are multiple c++ libraries that performs CSR SpMV multiplications like:

- cuSPARSE [1], the official library supported by Nvidia that can perform multiplication between matrices and vectors in different formats with CUDA.
- Ginkgo [2], like MAGMA offers high-performance solution for multicore linear algebra algorithms for different platforms.

IV. METHODOLOGY AND CONTRIBUTIONS

A. Dynamic algorithm

To reduce the unbalance

- 1) The row array is splitted in equal subsection for different blocks using again the ceiling division.
- 2) For each block a variable in the shared memory *pendingRow* is initialized and used to contain the next row number that is waiting for the assignment.
- 3) Each thread uses the function *atomicAdd(pendingRow, 1)* to increment the value of *pendingRow* and, at the same time, read its content
- 4) Using the value obtained each thread computes the products and the accumulation of the entire row and repeats step 3 and 4 until the block reach the end of its subsection.

Algorithm 1 Dynamic SpVM algorithm

Require: input, output and matrix structures.

```

1: procedure VMCSR_MUL(output, matrix, input)
2:   blockStart, blockEnd      ▷ Ceiling division
3:   pendingRow = 0              ▷ (only thread 0);
4:   syncthreads()
5:   while true do
6:     pendingRow = atomicAdd(pendingRow, 1)
7:     + blockStart
8:     if pendingRow > blockEnd then return
9:     end if
10:    startRow = matrix.rowArray[pendingRow]
11:    endRow = matrix.rowArray[pendingRow + 1]
12:    acc = 0
13:    while startRow < endRow do
14:      col = matrix.colArray[startRow]
15:      acc + =
16:      matrix.dataArray[startRow]
17:      * input.dataArray[col]
18:      startRow + +
19:    end while
20:    output.dataArray[pendingRow] = acc
21:  end while
22: end procedure

```

This dynamic assignment for threads in the same block implemented in the algorithm ?? tries to mitigate the unbalanced workload introduced in some rows. In facts, if some thread spend too much time when computing a single row, other available threads may perform also rows that in a static division should be reserved for others.

B. Optimized Dynamic algorithm

The dynamic algorithm can be improved further using a better organization of workload between threads. More performance can then be obtained by:

- Dividing the workload between blocks using an approach based on the number of not null elements instead the

rows. To achieve this, the ceiling division can be performed on the not null array instead on the row array and a binary search can be performed to find the relative rows boundaries.

- Assigning a warp for each row instead a single thread. In this way, because 32 threads are assigned per row, longer rows can be processed in shorter time and the partials obtained by threads in the same row can be reduced using more *__shfl_down_sync* functions.
- Assigning the read only cache on the input vector to improve the speed of the algorithm by leveraging on the data reusage.

Algorithm 2 Kernel used for the preliminar block division. The kernel is launched with the same number of blocks of the main kernel 3 but only one thread per block.

Require: matrix structure and the preinitialized *blockDivision* array on the GPU's global memory.

```

1: procedure VMCSR_DIV(matrix, blockDivision)
2:   blockStart, blockEnd  ▷ Ceiling division on the not
                           null element's array
3:   ...                    ▷ output vector initialization phase
4:   if blockIdx.x == 0 then
5:     blockDivision[0] = 0
6:     blockDivision[blockDim.x - 1] = rowSize
7:     return
8:   end if
9:   row = -1
10:  left = 0
11:  right = rowSize
12:  while left <= right do
13:    mid = (left + right) / 2
14:    if rowArray[mid] = blockStart then
15:      row = mid
16:      left = mid + 1
17:    else
18:      right = mid - 1
19:    end if
20:  end while
21:  blockDivision[blockIdx.x] = row
22: end procedure

```

C. Shared cooperative algorithm

The algorithm 3 does not adds a significant improvement for heavily unbalanced matrices. Another approach that tries to handle also those cases uses the shared memory to contain three different buffers:

- *sharedBuffer*, a space used as buffer to contain the result of the multiplication between elements of the matrix and the input vector (each thread operates on around 14 elements).
- *loadedRows*, a space of *blockSize.x* cells used as cache for parts of the CSR row vector (each threads loads one pointer).

Algorithm 3 Optimized dynamic algorithm

Require: matrix structure, block division array, input vector and output vector structures.

```
1: procedure VMCSRMUL(output, matrix, input,
   blockDivision)
2:   blockStart = blockDivision[blockIdx.x]
3:   blockEnd = blockDivision[blockIdx.x + 1]
4:   pendingRow = 0  $\triangleright$  (only thread 0);
5:   syncthreads()
6:   row = RowElection
7:   while row < blockEnd do
8:     startRow = matrix.rowArray[row]
9:     endRow = matrix.rowArray[row + 1]
10:    acc = 0
11:    while startRow < endRow do
12:      idx = min(startRow + laneId, endRow)
13:      col = matrix.colArray[idx]
14:      acc +=
15:        (matrix.dataArray[idx]
16:         * __ldg(input.dataArray[col]))
17:        *(startRow + laneId < endRow)
18:      startRow ++
19:    end while
20:    acc = ReduceSum
21:    if laneId == 0 then
22:      output.dataArray[row] = acc
23:    end if
24:    row = RowElection
25:  end while
26: end procedure
```

Algorithm 4 Row election System

```
1: procedure ROWELECTION
2:   if laneId == 0 then
3:     row = atomicAdd(pendingRow, 1)
4:     +blockStart
5:   end if
6:   __shfl_sync(0xffffffff, row, 0)
7: end procedure
```

Algorithm 5 Warp reduction system

```
1: procedure REDUCESUM(acc)
2:   acc = __shfl_down_sync(0xffffffff, acc, 16)
3:   acc = __shfl_down_sync(0xffffffff, acc, 8)
4:   acc = __shfl_down_sync(0xffffffff, acc, 4)
5:   acc = __shfl_down_sync(0xffffffff, acc, 2)
6:   acc = __shfl_down_sync(0xffffffff, acc, 1)
7: end procedure
```

- *rowsBuffer*, a space of $blockSize.x - 1$ used for accumulating partials for a small amount of rows (almost each thread maintains one cell).

Using these three different shared spaces, each block performs these steps:

- 1) The entire block cooperates to prepare the shared memory for processing $blockSize - 1$ rows by initializing with zeroes the *rowsBuffer* space and loading the necessary CSR pointers from the row array in the *loadedRows* space.
- 2) All threads load a certain amount of the required elements from the matrix and the input vector in a fully coaleshed way, perform the products and save the result in *sharedBuffer*.
- 3) The loaded elements are divided then in equal strides and the threads perform binary searches on the *loadedRows* space to identify the correspondent rows of the first elements of the strides. Then the strides are reduced in partials and accumulated in the *rowsBuffer* using *atomicAdd()*. In this phase, a thread can eventually operate in multiple rows.
- 4) The step 2 and 3 are performed until all not null elements of all rows' subset are processed.
- 5) Each threads stores one result from the *rowsBuffer* to the output vector in the global memory.
- 6) All steps from 1 to 5 are repeated until all rows of the matrix are processed.

The procedure 6 shows a heavily simplified representation of the shared cooperative algorithm.

Algorithm 6 Shared cooperative algorithm

Require: matrix structure, block division array, input vector and output vector structures.

```
1: procedure VMCSRMUL(output, matrix, input,
   blockDivision)
2:   blockStart, blockEnd  $\triangleright$  (From binary search kernel).
3:   sharedBuffer, rowsBuffer, loadedRows
4:   for (blockDim.x - 1) steps in [blockStart, blockEnd]
   do
5:     if threadIdx.x in rows block then
6:       Row space initialization
7:     end if
8:     for sharedBufferDim steps in blockNotNulls
   do
9:       cooperative multiplication on shared buffer
10:      Stride division of shared Buffer
11:      Binary search of first lane element
12:      Stride reduction
13:    end for
14:    if threadIdx.x in rows block then
15:      Rows unloading to global
16:    end if
17:  end for
18: end procedure
```

V. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP

A. Dataset description

A dataset composed by 6 matrices is used for testing and benchmark all four algorithms. These matrices contains a

number of not null elements between 1 and 1,5 million of not null elements and they have different organization of the elements and dispersion that challenges the algorithms in different ways: The used dataset is obtained from the SuiteSparse Matrix Collection [3], a website that contains sample matrices which can be used for benchmarking algorithms.

Dataset	Dimension (R x C)	Non-zero entries	Type
dbir2 [4]	18,906 x 45,887	1,158,159	Unbalanced rows
ex11 [5]	16,614 x 16,614	1,096,948	Diagonally distributed (low density)
language [6]	399,130 x 199,130	1,216,334	Upper triangular
Linux_call_graph [7]	324,085 x 324,085	1,208,908	Dense matrix (worst case)
nemeth24 [8]	9,506 x 9,506	1,506,550	Diagonally distributed (best case)
twotone [9]	120,750 x 120,750	1,206,265	Diagonally distributed (moderate dispersion)

TABLE I
STRUCTURAL PATTERNS OF THE BENCHMARK SPARSE MATRICES.

B. System Description

The system used for the execution of the benchmarks is uses CUDA 12.3.2, GCC 11.5.0, and is composed by a Intel(R) Xeon(R) Silver 4214 CPU and A Nvidia A30 GPU.

Characteristic	Value	Device
Global memory	24 GB	GPU: Nvidia A30
Theoretical BW	933 GB/s	
CUDA Cores	10752	
SM Number	108	
FP32 Performance	10.3 TFLOPS	
Cores number	12	CPU: Intel(R) Xeon(R) Silver 4214
Threads number	24	
Clock speed	2.2Ghz	

TABLE II
BENCHMARK SYSTEM SPECS

C. Experimental Set-up

The performance is measured using the execution time elapsed measured with *cudaEventRecord* functions for CUDA algorithms.

D. Block size estimation

The parameters used for launching the kernels are calculated empirically in base of the amount of shared memory available per block, the number of cuda cores per SM and the size of the input matrix. In particular:

- The number of threads per block is equal to the number of CUDA cores per SM divided by the available shared memory per block in MB (*cores/sharedPerSM*).
- The number of blocks is equal to the number of not null elements of the input matrix divided by a constant obtained empirically of around 3328 (*notNull/NotNullPerBlock*).
- In the kernel 6, the dimension of the shared memory is related on the number of elements that each thread should load in the *sharedBuffer* (the best performances are obtain for 14 elements per tread).

VI. EXPERIMENTAL BENCHMARKS

The following tables shows the values of GFLOPs/s (III) and effective bandwidth (IV) obtained from benchmarking on 256 blocks of 512 using float values. The figure 2 gives also a visualization of table III.

Matrix	Parallel 1	Parallel 2	Parallel 3	Sequential
dbir2 [4]	152,93	142,78	34,8	1,69
ex11 [5]	71,44	71,74	21,02	0,38
language [6]	31	82,4	1,91	0,32
Linux_call_graph [7]	0,85	1,72	2,51	0,25
nemeth24 [8]	43,4	50	16,81	0,37
twotone [9]	44,42	51,84	4,7	0,42

TABLE III
GFLOPs/S COMPARISON BETWEEN PARALLEL AND SEQUENTIAL IMPLEMENTATIONS

Matrix	Parallel 1	Parallel 2	Parallel 3
dbir2 [4]	932,56	870,66	212,49
ex11 [5]	435,15	437	128,03
language [6]	250,37	656,65	15,27
Linux_call_graph [7]	6,52	13,11	19,12
nemeth24 [8]	263,8	305,51	102,18
twotone [9]	292,8	241,75	31,03

TABLE IV
BANDWIDTH (GB/s) COMPARISON BETWEEN PARALLEL IMPLEMENTATIONS

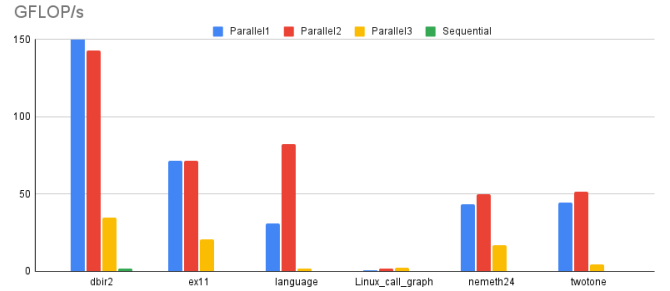


Fig. 2. Performance differences between the four algorithms

VII. RESULTS AND CONCLUSION

The results indicates that:

- The best situation is when the not null elements in the matrix are concentrated on the diagonal.
- If the number of not null elements per row is almost uniform for each row in the matrix, the first and the second parallel algorithms give similar results.
- The second parallel algorithm behaves better in different situations than the others implementations thanks to its load distribution system.
- The third parallel algorithm is more performant than the others only if the number of not null elements per row is very large and the dispersion of the matrix is high.

Also the block size affects the performances of the algorithms. In facts:

- In the first implementation the block size does not affect much the performances because the static allocation of workload.
- In the second implementation a lower block size may reduce the concurrency on the assignment system of the rows.
- In the third implementation a block size that is much lower than the average number of not null elements per row is more convenient because reduces significantly the concurrency introduced when accumulating the result on the output vector.

REFERENCES

- [1] NVIDIA. `cusparse`. [Online]. Available: <https://developer.nvidia.com/cusparse>
- [2] Ginkgo. [Online]. Available: <https://ginkgo-project.github.io>
- [3] U. of Florida. Suitesparse matrix collection. [Online]. Available: <https://sparse.tamu.edu/>
- [4] Meszaros. `dbir2` matrix. [Online]. Available: <https://sparse.tamu.edu/Meszaros/dbir2>
- [5] FIDAP. `ex11` matrix. [Online]. Available: <https://sparse.tamu.edu/FIDAP/ex11>
- [6] Tromble. `language` matrix. [Online]. Available: <https://sparse.tamu.edu/Tromble/language>
- [7] Sorensen. `Linux_call_graph` matrix. [Online]. Available: https://sparse.tamu.edu/Sorensen/Linux_call_graph
- [8] Nemeth. `nemet24` matrix. [Online]. Available: <https://sparse.tamu.edu/Nemeth/nemeth24>
- [9] ATandT. `twotone` matrix. [Online]. Available: <https://sparse.tamu.edu/ATandT/twotone>